DATA-260 Homework 8

DATA-260 Homework 8

GitHub repository: https://github.com/shrustishetty-2401/data260-9004

Student and Configuration

- Student: Shrusti Shetty
- SID4: 9004
- PORT_BASE: 8004
- PREFIX: `s9004`
- SEED: 9004
- VERIFY_SEED: 269004
- DOMAIN_ID: 4
- Hardware: MacBook Pro, Apple Silicon
- Local model: `qwen3:8b`
- Final branch: `hw8`
- Final tag: `hw8`
- Final tagged commit: `1c8cb440841a2fa223a0ba7e2f9d315fdf77c5df`

Part 1: Kafka Vulnerability Request/Response Service

The primary domain entity is `Vulnerability`.

The worker stores entities in an in-memory dictionary. Clients communicate with the worker only through Kafka.

Kafka Topics

- `s9004_vulnerability_requests`
- `s9004_vulnerability_responses`

Request Schema

```
{
  "correlation_id": "unique-request-id",
  "reply_to": "s9004_vulnerability_responses",
  "operation": "CREATE_VULNERABILITY",
```

```
  "data": {
    "vulnerability_code": "HW8-DEMO-001",
    "vulnerability_title": "Kafka demonstration vulnerability",
    "package_name": "package-0",
    "submitter_email": "student@example.com",
    "description": "A sufficiently long vulnerability description.",
    "category": "Input Validation",
    "terms_accepted": true
  }
}
```

Response Schemas

Successful CREATE responses return the generated UUID:

```
{
  "correlation_id": "same-request-id",
  "success": true,
  "entity_id": "uuid",
  "message": "Entity created"
}
```

Successful GET responses return the complete entity:

```
{
  "correlation_id": "same-request-id",
  "success": true,
  "entity": {
    "entity_id": "generated-uuid",
    "vulnerability_code": "HW8-DEMO-001",
    "vulnerability_title": "Kafka demonstration vulnerability",
    "package_name": "package-0",
    "submitter_email": "student@example.com",
    "description": "A sufficiently long vulnerability description.",
    "category": "Input Validation",
    "terms_accepted": true
  }
}
```

Error responses preserve the correlation ID and include an error message:

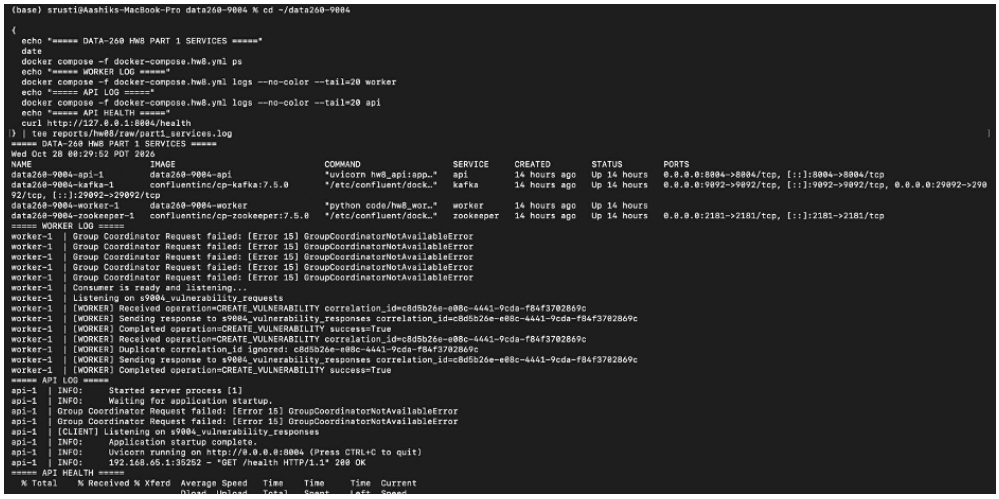
```
{
  "correlation_id": "same-request-id",
  "success": false,
  "error": "Entity not found"
}
```

Functionality

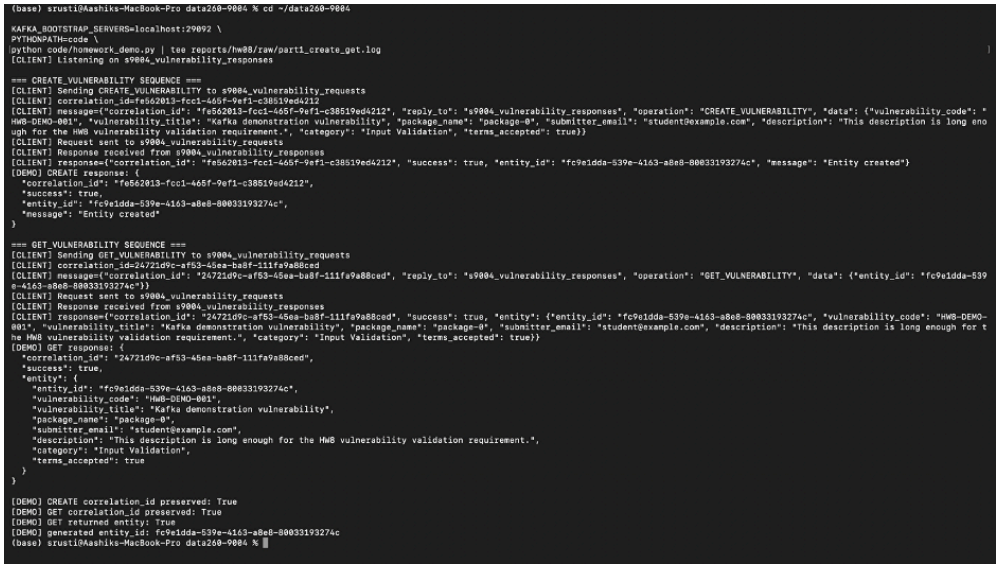
- CREATE vulnerability requests
- GET vulnerability requests

- UUID entity identifiers
- Correlation IDs preserved in every response
- Input validation
- Unknown-operation errors
- Missing-entity errors
- Duplicate correlation handling

The CREATE then GET demonstration succeeded. The GET request returned the created vulnerability entity, and correlation IDs were preserved.



Part 1 service evidence



Part 1 CREATE and GET evidence

Evidence:

- `raw/part1_create_get.log`
- `raw/part1_error_tests.json`
- `raw/part1_services.log`

Part 2: Multi-Agent Kafka Pipeline

Implemented separate Planner, Writer, and Reviewer processes.

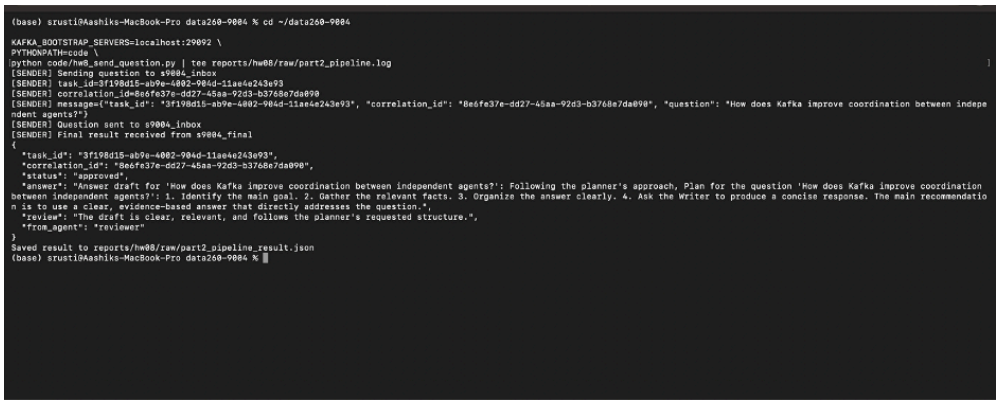
Kafka Topics

- `s9004_inbox`
- `s9004_tasks`
- `s9004_drafts`
- `s9004_final`

Agent Responsibilities

- Planner receives the question and creates a plan.
- Writer creates an answer draft.
- Reviewer evaluates the draft.
- Approved results are published to the final topic.

The pipeline completed successfully with status `approved`. The task ID and correlation ID were preserved through the pipeline.



Part 2 pipeline evidence

Evidence:

- `raw/part2_pipeline_result.json`
- `raw/part2_pipeline.log`
- `RUN_LOG.txt`

Part 3: Reliability, Failure Handling, and Ordering

Duplicate Delivery and Idempotency

The same CREATE request was delivered twice. Both responses returned the same entity ID.

- `same_entity_id`: `true`
- `duplicate_prevented`: `true`

Evidence:

- `raw/duplicate_delivery.json`

Consumer Death and Recovery

The Writer was stopped during a simulated long-running task. After restarting the Writer, Kafka redelivered the uncommitted message and the task completed successfully.

- `recovered_after_restart`: `true`
- `draft_received`: `true`

Evidence:

- `raw/consumer_death.json`

Poison Messages and Dead-Letter Handling

Poison messages intentionally failed processing. The Writer retried each message three times and then moved it to the dead-letter topic.

- Retry limit: 3
- Dead-letter topic: `s9004_dead_letter`
- `moved_to_dead_letter`: `true`

Evidence:

- `raw/poison_message.json`

Kafka Ordering

Twenty unkeyed and twenty keyed messages were tested.

Case	Partitions	Order preserved
Unkeyed messages	2	No
Keyed messages	1	Yes

The ordering experiment used two Kafka partitions and two Writer instances in the same consumer group. Unkeyed messages were distributed across partitions and were not globally ordered. Keyed messages used the same entity key, were routed to one partition, and preserved their order.

- `keyed_order_preserved`: `true`
- Verification seed: `269004`

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

python - <<'PY' | tee reports/hw08/raw/part3_failure_summary.log
import json
from pathlib import Path

raw = Path("reports/hw08/raw")

duplicate = json.loads(
    (raw / "duplicate_delivery.json").read_text()
)
death = json.loads(
    (raw / "consumer_death.json").read_text()
)
poison = json.loads(
    (raw / "poison_message.json").read_text()
)
ordering = json.loads(
    (raw / "ordering_results.json").read_text()
)

print("===== DUPLICATE DELIVERY =====")
print("same_entity_id:", duplicate["same_entity_id"])
print("duplicate_prevented:", duplicate["duplicate_prevented"])

print("\n===== CONSUMER DEATH =====")
print("recovered_after_restart:", death["recovered_after_restart"])
print("draft_received:", death["draft_received"])

print("\n===== POISON MESSAGE =====")
print("retry_count:", poison["retry_count"])
print("dead_letter_topic:", poison["dead_letter_topic"])
print("moved_to_dead_letter:", poison["moved_to_dead_letter"])

print("\n===== ORDERING =====")
print(
    "unkeyed partitions:",
    ordering["unkeyed"]["partitions_seen"],
)
print(
    "unkeyed received order:",
    ordering["unkeyed"]["received_sequences"],
)
print(
    "keyed partitions:",
    ordering["keyed"]["partitions_seen"],
)
print(
    "keyed received order:",
    ordering["keyed"]["received_sequences"],
)
print(
    "keyed_order_preserved:",
    ordering["keyed_order_preserved"],
)
[PY
===== DUPLICATE DELIVERY =====
same_entity_id: True
duplicate_prevented: True

===== CONSUMER DEATH =====
recovered_after_restart: True
draft_received: True

===== POISON MESSAGE =====
retry_count: 3
dead_letter_topic: s9004_dead_letter
moved_to_dead_letter: True

===== ORDERING =====
unkeyed partitions: [0, 1]
unkeyed received order: [1, 2, 3, 7, 4, 5, 8, 10, 6, 9, 11, 12, 15, 16, 13, 14, 17, 18, 20, 19]
keyed partitions: [1]
keyed received order: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
keyed_order_preserved: True
(base) srusti@Aashiks-MacBook-Pro data260-9004 %
```

Part 3 failure and ordering evidence

Evidence:

- `raw/ordering_results.json`
- `raw/part3_failure_summary.log`
- `raw/ordering_writer_a.log`
- `raw/ordering_writer_b.log`

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004

KAFKA_BOOTSTRAP_SERVERS=localhost:29092 \
PYTHONPATH=code \
python code/hw8_writer.py
[WRITER] Listening on s9004_tasks
```

Writer A



Writer B



Ordering result

Kafka Reliability Conclusions

Kafka provides:

- Durable message storage
- Consumer-group coordination
- Redelivery of uncommitted messages
- Retry and dead-letter processing
- Correlation between requests and responses
- Per-key ordering through partition assignment
- Decoupling between independent services

The producer is responsible for publishing valid messages and using stable keys when ordering matters. The consumer is responsible for validation, idempotency, retries, offset commits, and dead-letter handling.

Comparison with HW2

HW2 used a simpler request/response workflow. HW8 extends that design with Kafka topics, independent consumers, consumer groups, offset commits, retries, dead-letter handling, idempotency, consumer restart recovery, and key-based ordering.

Kafka adds durable asynchronous coordination, but it also requires explicit handling of commits, retries, duplicate messages, consumer failures, and ordering.

Two benefits gained from Kafka were independent scaling of agents and decoupled failure recovery. Two new responsibilities were duplicate/idempotency handling and explicit ordering and offset management.

Failure Comparison Table

Failure scenario	Before fix	After fix
Duplicate delivery	Replay could create an additional side effect.	The same entity ID was returned and the duplicate was prevented.
Consumer killed mid-work	Processing stopped before the offset was committed.	Restart caused redelivery and successful completion.
Poison message	A bad message could continue failing without bounded handling.	The message was retried three times and moved to `s9004_dead_letter`.
Two partitions/two consumers	Unkeyed related messages were not globally ordered.	Keyed related messages used one partition and preserved order.

Each row is supported by the matching raw JSON or log evidence listed in the Part 3 sections above.

Short Implementation Evidence

The worker dispatches only the two required operations:

```
if operation == "CREATE_VULNERABILITY":
    response = create_vulnerability(data)
elif operation == "GET_VULNERABILITY":
    response = get_vulnerability(data)
else:
    response = {"success": False, "error": "Unknown operation"}
```

Every response preserves the request correlation ID:

```
response["correlation_id"] = message["correlation_id"]
await producer.send_and_wait(reply_to, json.dumps(response).encode())
```

Poison messages use bounded retries and then move to the dead-letter topic:

```
if retry_number >= MAX_RETRIES:
    await producer.send_and_wait(DEAD_LETTER_TOPIC, encoded_message)
    await consumer.commit()
```

Ordering uses the related entity as the Kafka message key:

```
key = b"vulnerability-42"
await producer.send_and_wait(TASKS_TOPIC, payload, key=key)
```

AI Use

1. AI assistance was used for drafting Kafka client, worker, multi-agent, retry, dead-letter, ordering, verification, and report code. The student ran the commands, inspected the outputs, captured the evidence, and made the final repository changes.
2. One initial AI-produced design created the Kafka producer outside an asynchronous function, which failed with a Python 3.13 event-loop error. Another initial retry design waited for Kafka redelivery instead of retrying poison messages immediately.
3. These problems were detected by running the Docker service and poison-message tests. The API failed during startup in the first case, and the poison test timed out in the second case.
4. The producer and consumer objects were moved into the asynchronous startup method. The Writer was changed to retry in a bounded loop, send the message to `s9004_dead_letter` after three failures, and commit the offset only after dead-letter publication.

Reproducibility Evidence

- `RUN_LOG.txt`
- `METRICS.md`
- `AI_USE.md`
- `verification.json`
- `raw/part1_create_get.log`
- `raw/part1_error_tests.json`
- `raw/part1_services.log`
- `raw/part2_pipeline_result.json`
- `raw/part2_pipeline.log`
- `raw/duplicate_delivery.json`
- `raw/consumer_death.json`
- `raw/poison_message.json`
- `raw/ordering_results.json`
- `raw/part3_failure_summary.log`

Final Verification

The verification script reported:

- API health: passed
- Part 1 CREATE/GET evidence: passed
- Part 1 error tests: passed
- Part 2 pipeline approval: passed
- Duplicate delivery prevention: passed
- Consumer death recovery: passed
- Poison dead-letter handling: passed
- Keyed ordering preservation: passed
- Required evidence directory: passed

Overall result: `all_passed: true`

Submission Files

The final report PDF is:

`reports/hw08/Shetty_HW8.pdf`

The final submission includes the implementation, Docker/Kafka configuration, tests, screenshots, raw JSON results, logs, metrics, AI-use documentation, verification output, and the final PDF.